Haker Jasio (aaa)

Limit pamięci: 256 MB Limit czasu: 1.00 s

Jasio chce udowodnić swoim rodzicom, że naprawdę robi postępy z języka polskiego (a nie tylko zajmuje się informatyką). Niestety jego oceny z całego semestru nie wyglądają najlepiej. Jasio użył więc swoich hakerskich umiejętności i włamał się do elektronicznego dziennika, aby poprawić sytuację – niestety ma dosyć ograniczone możliwości manipulacji.

W dzienniku zapisano oceny Jasia z kolejnych lekcji w postaci ciągu A_i. Jasio chce, aby ten ciąg miał jak najdłuższy podciąg rosnący. Formalnie, interesuje go największe możliwe K, dla którego istnieją indeksy $1 \leq i_1 < i_2 < \cdots < i_k \leq N$ takie, że

$$A_{i_1} < A_{i_2} < \cdots < A_{i_k}$$

Jasio może oszukać tylko raz. Wolno mu wybrać jeden spójny przedział ocen oraz jedną liczbę całkowitą D i dodać D do każdej oceny w tym przedziale.

Nikt raczej nie zauważy takiej drobnej ingerencji, ale Jasio liczy, że pozwoli to znacząco wydłużyć najdłuższy rosnący podciąg.

Twoim zadaniem jest obliczyć, jaka jest **największa** możliwa długość rosnącego podciągu po wykonaniu **dokładnie jednej** takiej operacji.

Wejście

W pierwszym wierszu wejścia znajdują się dwie liczby całkowite N i X, oddzielone pojenynczym odstępem i oznaczające kolejno długość ciągu ocen Jasia oraz ograniczenie na parametr D ($-X \leq D \leq X$).

Wyjście

W pierwszym (jedynym) wierszu wyjścia powinna się znaleźć jedna liczba całkowita, oznaczająca największą możliwą długość rosnącego podciągu po wykonaniu dokładnie jednej operacji.

Ograniczenia

- $1 \leq N \leq 200\,000$.
- $0 \leq X \leq 10^9$.
- $1 \leq A_i \leq 10^9$.

Podzadania

Podzadanie	Warunki	Punkty
1	$N, X \leq 10$	5
2	$N, X \leq 50$	10
3	$N \leq 1000$	13
4	$X = 0$	10
5	$N \leq 50000, X \leq 15$	20
6	$X = 10^9$	17
7	brak dodatkowych ograniczeń	25

Przykład

Wejście	Wyjście	Wyjaśnienie

```
8 10
7 3 5 12 2 7 3 4
```

```
5
```

Jasio może wybrać interwał $[2, 3]$ oraz $D = -5$, co spowoduje obniżenie ocen A_2 oraz A_3 o 5. Nowy ciąg będzie wyglądał następująco: $(7, -2, 0, 12, 2, 7, 3, 4)$, gdzie można znaleźć najdłuższy podciąg rosnący $(-2, 0, 2, 3, 4)$. Długość tego podciągu to 5.

Ruletka (bbb)

Limit pamięci: 16 MB Limit czasu: 1.00 s

To zadanie jest interaktywne. Po wypisaniu każdego wiersza należy opróżnić bufor. W C++ możesz użyć **cout « flush**, w Pythonie – **sys.stdout.flush()**. Należy ściśle trzymać się protokołu opisanego w sekcji *Interakcja* – niewczytanie wiadomości wysłanej przez interaktor skutkuje werdyktem *wrong answer* lub innym.

Krzyś miał naprawdę dobry dzień. Po tygodniu programowania bez snu, hektolitrach wypitej kawy i tysiącach linii kodu wreszcie udało mu się naprawić błąd w kodzie firmy, w której pracuje. W nagrodę dostał sporą wypłatę ... w bajtcoinach. Wreszcie! Jego wirtualny portfel wypełnił się cyfrową walutą.

Ale każdy programista ma swój słaby punkt. U Krzysia nie był to brak średników, nie był to też wątpliwej jakości kod. Jego słabością było kasyno. Nie takie zwykłe – kasyno pełne dziwnych gier, intrygującej muzyki, niezwykłych trunków, a co najważniejsze: stworzone przez informatyków.

Dlatego, gdy jego kumpel Damian zadzwonił i rzucił hasło: „Ej, Krzychu, chodź do kasyna, musisz zobaczyć nową ruletkę – podobno wymyślił ją sam twórca bajtcoinów, TSG!", Krzyś nawet się nie wahał.

W kasynie czekała na nich ona: ruletka 6.7. Ulepszona. Nietypowa. Trochę podejrzana. Ale za to z nagrodą absolutnie nie do odrzucenia.

Ruletka miała 256 pól ułożonych w koło. Dla $1 \leq i \leq 255$, pola i oraz $i+1$ sąsiadowały ze sobą, podobnie pola 256 i 1. I co najciekawsze: gdzieś tam, między wszystkimi liczbami, skrywało się jedno jedyne pole wygrywające.

Celem rozgrywki było odgadnięcie pola wygrywającego. W każdej turze Krzyś mógł wskazać kilka pól, a specjalna maszyna odpowiadała mu, jaka jest najmniejsza odległość (po okręgu) między którymkolwiek z wybranych pól a polem z nagrodą. Oczywiście – każda próba kosztowała bajtcoiny: $\left\lceil \frac{256}{k} \right\rceil$ bajtcoinów, gdzie k to liczba wskazanych pól. Innymi słowy, jest to najmniejsza liczba całkowita nie mniejsza niż $\frac{256}{k}$. Krzyś co prawda niedawno otrzymał sporą wypłatę w bajtcoinach, ale jak to u niego bywa, szybko się rozeszła – teraz zostało mu jedynie 512 bajtcoinów.

Damian spojrzał na niego z uśmiechem i powiedział: „Ta ... bo ty dasz radę."

Krzysiu odrzekł: „Ja nie dam rady?"

Krzysiu nie miał pomysłu jak zabrać się za tę grę, ale znał osobę, która jest dobra w takie gry i poprosił cię o pomoc. Teraz wszystko zależy od ciebie. Pomóż Krzysiowi odnaleźć wygrywające pole, nie wydając nie za dużo bajtcointów. Krzysiu będzie w pełni usatysfakcjonowany, jeśli wydasz **nie więcej niż 6 bajtcoinów, częściowo ucieszy go jednak dowolna wygrana**.

Interakcja

Kasyno wyjątkowo nie oszukuje: **numer wygrywającego pola jest ustalony na początku** i nie zależy od wykonywanych przez Krzysia strzałów.

Należy wykonać pewną liczbę zapytań. Każde z nich polega na wybraniu pewnej wartości k oraz pewnych k pól.

Po zadaniu zapytania postaci `? k` a_1 a_2 a_3 ... a_k gdzie k to liczba wybranych pól, a $a_1, a_2, ..., a_k$ to numery tych pól, należy wczytać jedną liczbę całkowitą – najmniejszą odległość między którymkolwiek z wybranych pól a polem wygrywającym.

Na końcu należy wypisać wykrzyknik oraz jedną liczbę – pozycję pola wygrywającego: `! x`

Pamiętaj, aby kolejne zapytania oddzielić białym znakiem!

Ograniczenia

Liczba pól w rozważanej ruletce to 256.

Podzadania

Podzadanie	Pozycja x pola wygrywającego	Punkty
1	$1 \leq x \leq 3$	24
2	$1 \leq x \leq 128$	33
3	$1 \leq x \leq 256$	43

x - pole wygrywające

Liczba C wydanych bajtcoinów	Procent punktów za daną grupę testów
$C \leq 6$	100
$6 < C \leq 512$	$100 - 5 \cdot \log_2(C - 5)$
$512 < C$	0

Przykładowa interakcja

Wejście	Wyjście
	`? 3 1 2 3`
`0`	
	`? 2 2 3`
`0`	
	`? 1 3`
`1`	
	`! 2`

Narzędzia do testowania lokalnego i testy przykładowe

W zadaniu są 3 testy przykładowe, każdy z nich reprezentuje inne podzadanie. Zostaną również udostępniony program **interaktor.cpp** oraz przykładowe błędne rozwiązanie o poprawnym schemacie komunikacji **abc.cpp**. Żeby uruchomić rozwiązanie na teście, należy skompilować rozwiązanie i interaktorkę, a następnie użyć komendy:

`[plik wykonywalny z interaktor.cpp] [test] [plik wykonywalny z rozwiązaniem]`

Na przykład: `./interaktor 0 ./abc`

Złoty Banan (ccc)

Limit pamięci: 256 MB Limit czasu: 1.00 s

W samym sercu dżungli dojrzewa Złoty Banan – legendarny owoc, który według małp przynosi szczęście każdemu, kto go zdobędzie. Niestety rośnie tak wysoko, że żadna małpa nie jest w stanie dosięgnąć go sama.

Małpy postanowiły więc połączyć siły i stworzyć "żywą wieżę", wskakując sobie na barana. Jednak nie wszystkie N małp ważą tyle samo, a każda chce koniecznie uczestniczyć w zdobyciu skarbu.

Jeśli dużo cięższa małpa wskoczy na plecy znacznie lżejszej, ta dolna od razu się przewraca i cała konstrukcja ląduje w krzakach. Dlatego obowiązuje zasada: nie wolno postawić małpy A **bezpośrednio** na małpie B, jeśli A jest **cięższa** od B o więcej niż ustalona wartość D.

Twoim zadaniem jest policzyć, na ile sposobów małpy mogą ustawić się jedna na drugiej, tak aby stworzyć stabilną wieżę i sięgnąć po Złoty Banan. Ponieważ możliwości może być bardzo dużo, należy podać **resztę z dzielenia** wyniku przez $10^9 + 9$.

Wejście

W pierwszym wierszu wejścia znajdują się dwie liczby naturalne N i D, oddzielone pojedynczym odstępem i oznaczające kolejno: liczbę małp oraz ustalony parametr D.

W drugim wierszu wejścia znajduje się N liczb naturalnych A_i, oznaczających wagi małp.

Wyjście

W pierwszym (jedynym) wierszu wyjścia powinna się znaleźć jedna liczba całkowita – reszta z dzielenia sposobów ustawienia małp przez $10^9 + 9$.

Ograniczenia

- $2 \leq N \leq 1\,000\,000$.
- $1 \leq D \leq 10^9$.
- $1 \leq A_i \leq 10^9$.

Podzadania

Podzadanie	Warunki	Punkty
1	$2 \leq N \leq 10$	10
2	$2 \leq N \leq 20$	21
3	$2 \leq N \leq 70$	20
4	$Wynik \leq 1\,000\,000$	19
5	brak dodatkowych ograniczeń	30

Przykład

Wejście

```
4 1
1 2 3 100
```

Wyjście

```
4
```

Wyjaśnienie

Trzy pierwsze małpy mogą ustawić się w każdej kolejności, poza $2, 1, 3$ i $1, 3, 2$. Ostatnia małpa musi być na samym dole.

Wejście

```
6 9
10 20 20 10 10 20
```

Wyjście

```
36
```

Wyjaśnienie

Małpa o wadze 20 nie może być nad małpą o wadze 10. Jest 6 sposobów na ustawienie małp o wadze 10, oraz 6 sposobów na ustawienie małp o wadze 20.